Line Renderer 2D Pro v1.1.0

A 2D pixel-perfect line strip renderer for Unity

Please visit https://linerenderer2d.com to see a better version of the documentation.

Making pixel-art games? This is for you!

Draw pixel-perfect 2D lines in the scene with any thickness and use them as if they were sprites. Lines are affected by layers, 2D lights, sorting layers and post-processing FXs. Add as many points as you need to create line strips and represent your ropes or cables, and update them every frame. Enable any of the visual effects available out of the box like dotted lines, textured lines, gradients, and more.

Compatibility: Windows and Mac, all versions of Unity since 2020.3.0.

Features

Fully integrated in the 2D URP scene, like a sprite

- Affected by 2D lights (can ignore lighting too).
- Each line renderer belongs to a sorting layer.
- It supports sprite masking (at material level).

Effects are applied per pixel distance, not per actual distance

- Starting at one endpoint of the line, color patterns progress one pixel at a time towards the other endpoint.

Thickness can behave in 2 different ways

- It can be constant on the screen disregarding the orthogonal size of the camera.
- Or it can adapt to it.

Reduced pixel vibration when moving the camera

- The camera's imaginary grid where the line is drawn does not exactly match the screen's grid, and the position of each point in the line does not coincide with the center of each cell in that grid either. That produces imprecisions when either points change their position or the camera moves.
- Adding some adjustments to the position of each point before drawing prevents that from happening and lets us move lines smoothly, no matter their thickness.

Editor preview of the line

- You can add, remove or move points of the line strip in the scene view, and see a preview of it, including its visual effects. It will not match the actual rendering of the line because it uses the editor's camera.

Visual Effects

Dotted lines

- You can choose the length, in pixels, of the dots and the length of the space between them.
- An offset can be applied to achieve interesting effects.
- When dotted lines use textures, colors are applied to each dot, in order, no matter the dot length.

Textured lines

- Define color sequences that will repeat along the entire line strip, starting at one endpoint.
- Color sequences can be stored in textures.

Color gradients

- Lines can be colored using a 2-colors gradient whose length and starting position can be defined as desired. Texture and gradient colors are mixed.
- Enable the adaptive gradient length if you want the color gradient to extend along the line regardless of its length.

Bounds offset

- A line strip is defined by a set of points, but you can choose at which pixel the line starts and how many pixels will be drawn from there.
- Changing the start point may affect the offset of the other visual effects, optionally.

Overlay texture

- A texture can be projected on the line. The position of the texture does not depend on the position of the line. This can be used to achieve the effect of a laser ray that goes through the smoke of a room, for example.

Usage

Just drag the provided prefab to the scene and you will see the preview in the Scene View. In order to render the actual line, you need to assign a camera to it by filling either the Camera field or the Camera Tag field before playing (implement the CameraResolver class to customize

how the camera is set in runtime). You can add new points to the Points list and move them using the position handlers in the Scene View. Be careful not setting the Maximum amount of points too low. Modify the Thickness of the lines as it fits better in your game. Adjust the Quad width factor accordingly: the greater thickness, the greater the width factor; otherwise the pixels of the line may look trimmed. Choose its sorting layer and take a look at the optimization tips.

Optimization Tips

The line renderer 2D is implemented with performance in mind. Apart from the built-ín techniques, it provides some ways to fine-tune its features to achieve the best performance for every project.

Quad width factor

It is a multiplier that affects the world-space “width” of the quads used as canvas to draw the lines. Most of the work in the GPU is done per-pixel so quads should be as thin as possible without trimming any pixel of the line.

Disabling auto-update

If the position of the points does not change every frame, you should disable the Auto Apply Position Changes option and call ApplyPointPositionChanges manually.

Keep background color transparent

Fully transparent pixels will be discarded, accelerating the drawing process. You can use a different color for debugging purposes.

Do not set any property if its value has not changed

Properties like Thickness, Start Point or Gradient Offset are all sent to the GPU in LateUpdate when any of them is set (so you can change them many times in the same frame without the cost of sending the values). You should check if the value to be set is the same that is already stored, so the line is not “marked as dirty”. This will avoid sending values when there is nothing to update.

Disable lighting

If the line should not be affected by lights, change the Is Lit flag in the material to save some GPU power.

Do not use more points than you need

Try to reduce the amount of points that form the line strip as much as possible, sometimes it looks just as good using fewer points.

Disable features

Every line visual effect can be disabled by leaving its corresponding checkbox in the material. Different shader variants are used for the different combinations.

Choosing maximum points

You can choose the size of the buffers that will be sent to and stored in the GPU, depending on the amount of points expected to form the line strip. The lower the maximum value, the better the performance. There is a fixed set of values for the Maximum Amount of Points property: 2, 32, 128 and Unlimited. The latter is by far more expensive than the others. Take into account that the value is stored in the material which means changing it in one line renderer will affect every line renderer that uses the same material.

Use a sprite if the line never moves

This may sound too obvious but, if a line is not expected to move ever, just manually draw it and use a sprite instead of the line renderer.